

EECE 306 Lab 4 notebook. The Quadrature Engine and Continuous Charge Distributions

Team 8. Thalea Collymore, Kroix Jones, Xavier Moore

Fill in every TODO, run `publish('Lab04_notebook.m')` from the toolbox root, print the HTML to PDF, three to five pages.

Contents

- [Setup](#)
- [Geometry before physics](#)
- [Observed order of each rule](#)
- [The Gauss rule does not fit the same mold](#)
- [Finite line against the infinite line formula](#)
- [Disk against the infinite sheet formula](#)
- [A nonuniform density through the same constructor](#)
- [Interpretation](#)
- [Problems encountered](#)
- [Full test suite](#)

Setup

```
clear; close all;
e0 = em.const.eps0();
```

Geometry before physics

The sum of the weights must return length, area, and volume with no field anywhere in sight.

```
circ = @(t)[cos(t) sin(t) zeros(size(t))];
[~, wc] = em.quad.line(circ, [0 2*pi], 400);
sph = @(th,ph)[sin(th)*cos(ph) sin(th)*sin(ph) cos(th)];
[~, ws] = em.quad.surf(sph, [0 pi], [0 2*pi], 60, 120);
ball = @(u,v,w)[w*sin(u)*cos(v) w*sin(u)*sin(v) w*cos(u)];
[~, wv] = em.quad.vol(ball, [0 pi], [0 2*pi], [0 1], 30, 60, 30);
sq = @(u,v)[u v 0];
[~, wq] = em.quad.surf(sq, [0 1], [0 1], 20, 20);
fprintf('circle length %.6f expect %.6f\n', sum(wc), 2*pi);
fprintf('sphere area %.6f expect %.6f\n', sum(ws), 4*pi);
fprintf('ball volume %.6f expect %.6f\n', sum(wv), 4*pi/3);
fprintf('square area %.6f expect %.6f\n', sum(wq), 1);
```

```
circle length 6.283185 expect 6.283185
sphere area 12.567806 expect 12.566371
ball volume 4.189541 expect 4.188790
square area 1.000000 expect 1.000000
```

Observed order of each rule

Errors are collected in a plain loop, then the standard fitting tool reports the observed order. One representative log log plot is shown.

```
Iex = exp(1) - 1;
Ns = [8 16 32 64 128]';
rules = {'midpoint', 'trapz', 'simpson'};
```

```

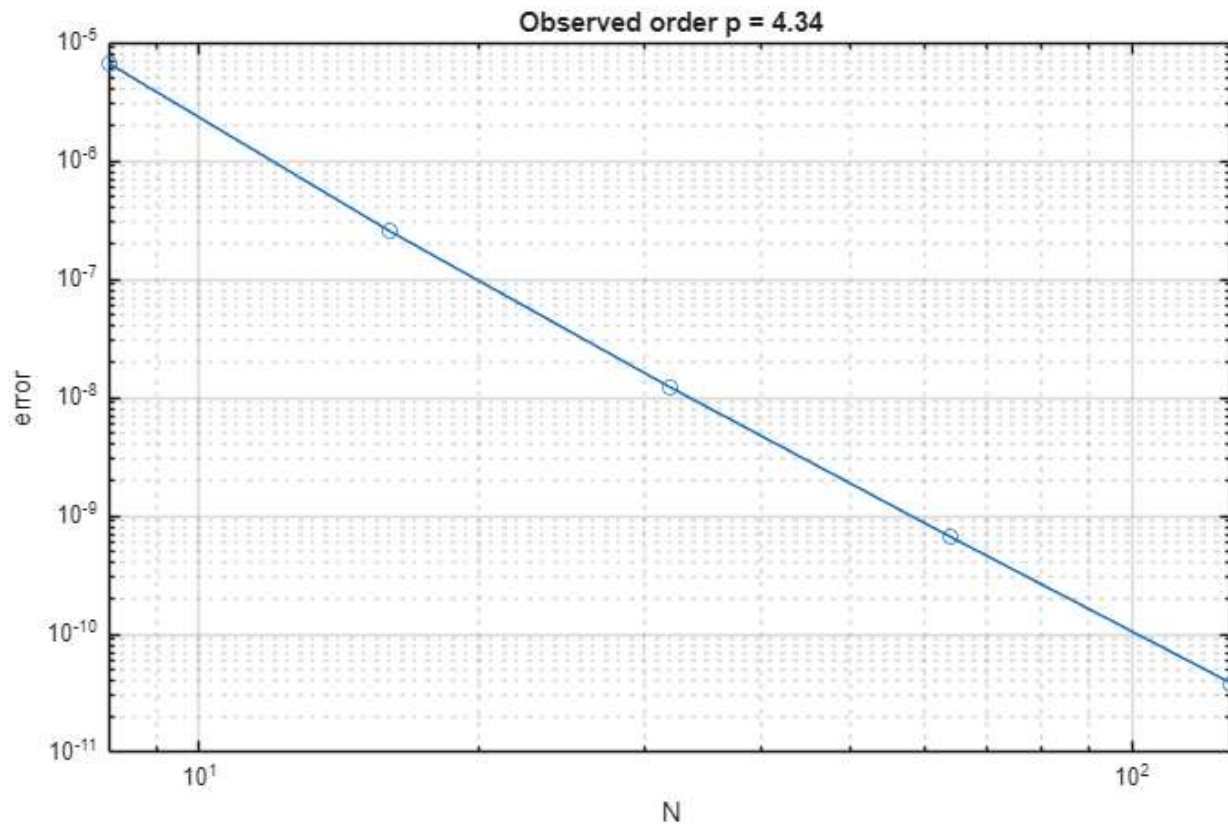
for k = 1:3
    rule = rules{k};
    errs = zeros(size(Ns));
    for j = 1:numel(Ns)
        [t, w] = em.quad.nodes(0, 1, Ns(j), rule);
        errs(j) = abs(sum(exp(t) .* w) - Iex);
    end
    p = em.test.convergence(@(N) errs(Ns == N), Ns, struct('plot', strcmp(rule, 'simpson')));
    fprintf('%-9s observed order p = %.2f\n', rule, p);
end

```

```

midpoint observed order p = 2.00
trapez observed order p = 2.09
simpson  observed order p = 4.34

```



The Gauss rule does not fit the same mold

```

for N = [2 4 8 16]
    [t, w] = em.quad.nodes(0, 1, N, 'gauss');
    fprintf('gauss N = %2d error = %.3e\n', N, abs(sum(exp(t) .* w) - Iex));
end

```

% The midpoint, trapezoid, and Simpson errors decrease at a predictable
 % power of N, so a straight line on a log-log plot makes sense for them.
 % The Gauss error decreases much faster and quickly reaches floating-point
 % precision. Because it does not follow one fixed power of N, a straight
 % line fit does not represent a meaningful single convergence order.

```

gauss N = 2 error = 3.855e-04
gauss N = 4 error = 9.330e-10

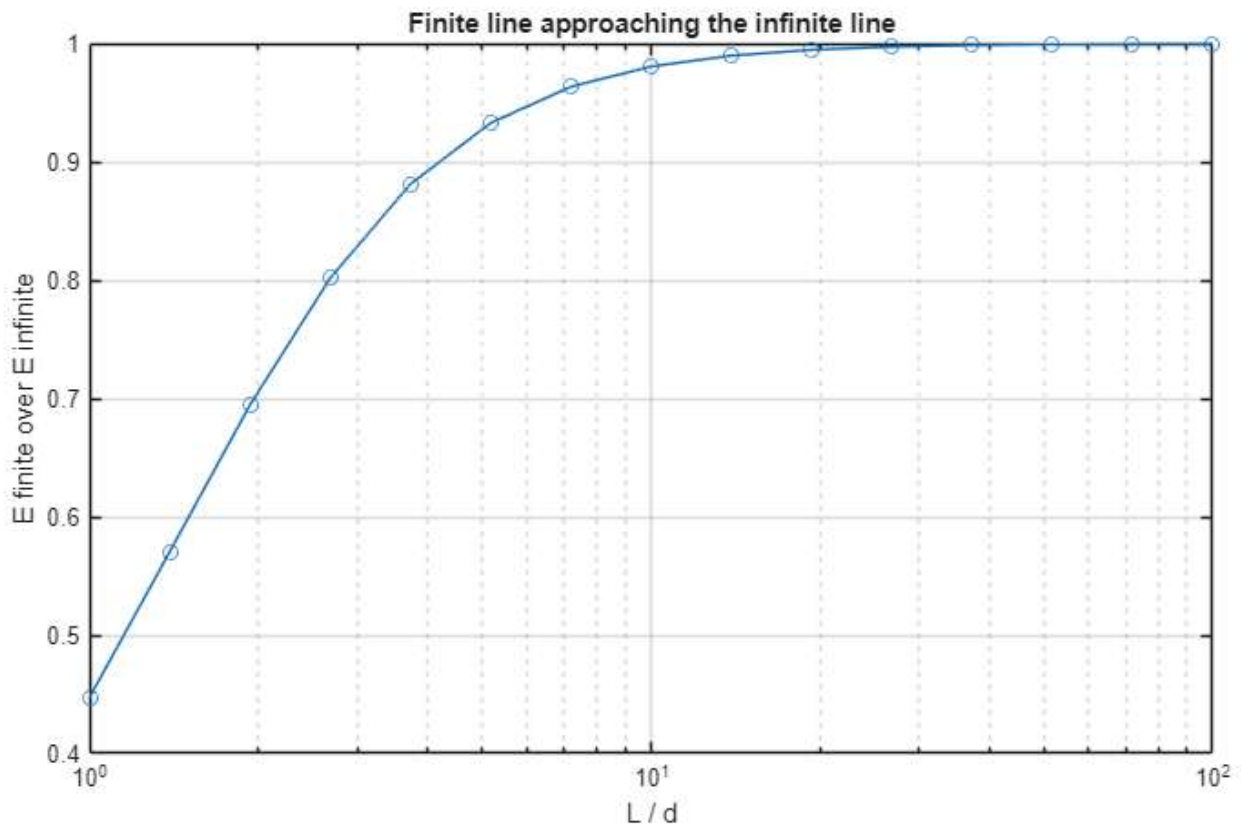
```

```
gauss N = 8   error = 6.661e-16
gauss N = 16  error = 2.887e-15
```

Finite line against the infinite line formula

```
d = 1;
Lod = logspace(0, 2, 15)';
ratio = zeros(size(Lod));
for k = 1:numel(Lod)
    L = Lod(k) * d;
    lineC = @(t)[zeros(size(t)) zeros(size(t)) t];
    sl = em.src.lineCharge(1e-9, lineC, [-L/2 L/2], 800);
    Ef = em.field.E(sl, [d 0 0]);
    ratio(k) = Ef(1) / (1e-9 / (2*pi*e0*d));
end
figure; semilogx(Lod, ratio, 'o-'); grid on
xlabel('L / d'); ylabel('E finite over E infinite')
title('Finite line approaching the infinite line')
iAbove = find(ratio > 0.99, 1);
fprintf('ratio first exceeds 0.99 at L/d about %.1f\n', Lod(iAbove));
```

ratio first exceeds 0.99 at L/d about 19.3



Disk against the infinite sheet formula

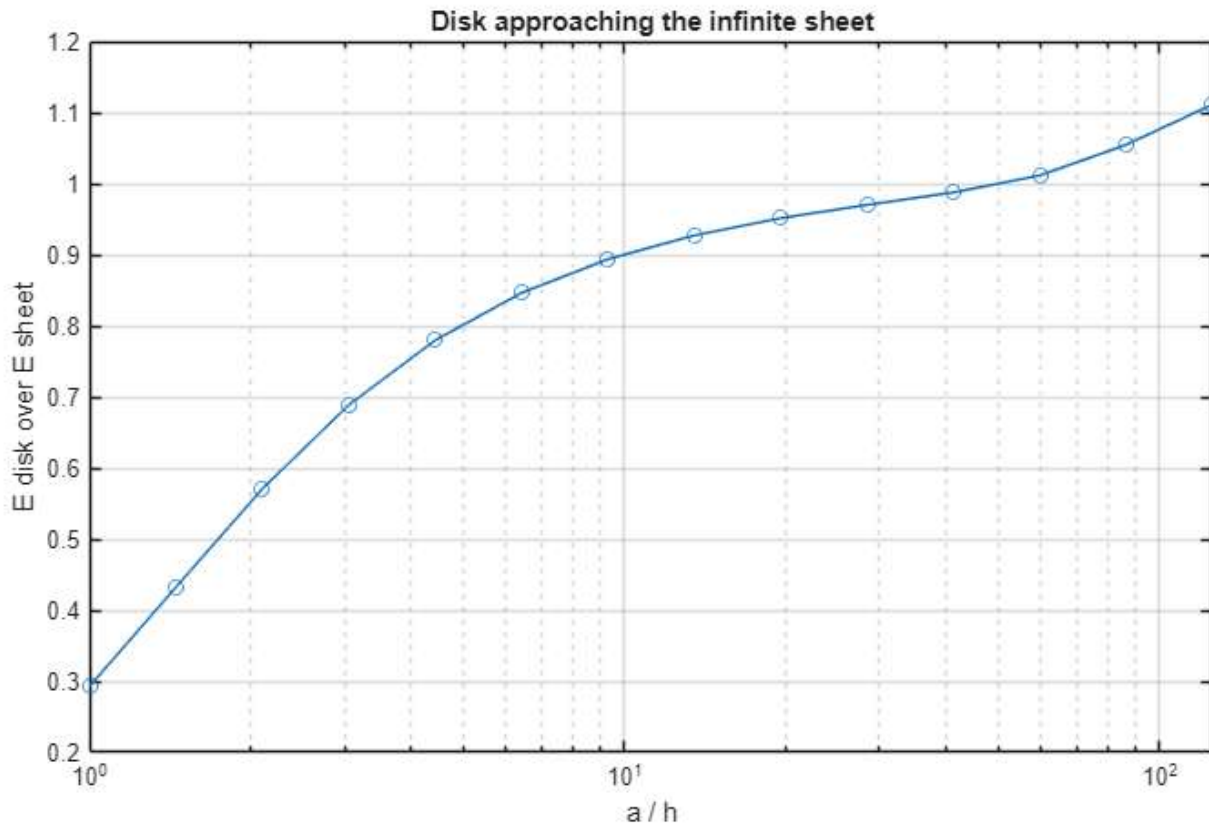
```
h = 1;
aoh = logspace(0, 2.1, 14)';
ratioD = zeros(size(aoh));
for k = 1:numel(aoh)
    a = aoh(k) * h;
```

```

disk = @(u,v)[u.*cos(v) u.*sin(v) zeros(size(u))];
sd = em.src.surfCharge(1e-9, disk, [0 a], [0 2*pi], 80, 80);
Ed = em.field.E(sd, [0 0 h]);
ratioD(k) = Ed(3) / (1e-9 / (2*e0));
end
figure; semilogx(aoh, ratioD, 'o-'); grid on
xlabel('a / h'); ylabel('E disk over E sheet')
title('Disk approaching the infinite sheet')
iA = find(ratioD > 0.99, 1);
fprintf('ratio first exceeds 0.99 at a/h about %.1f\n', aoh(iA));

```

ratio first exceeds 0.99 at a/h about 59.8



A nonuniform density through the same constructor

```

lineC = @(t)[zeros(size(t)) zeros(size(t)) t];
sn = em.src.lineCharge(@(r) r(:,3), lineC, [-1 1], 400);
fprintf('net charge of rho(z) = z on [-1, 1] = %.3e C (expect 0)\n', sum(sn.q .* sn.w));

```

net charge of $\rho(z) = z$ on $[-1, 1] = -2.602e-17$ C (expect 0)

Interpretation

The measured convergence orders were close to the expected values, which gives us confidence that the quadrature rules are working correctly. Measuring the order instead of assuming it is useful because an incorrect order can reveal a problem even when the numerical answers still look reasonable. For example, a mistake in node placement or in how a boundary is handled could cause the measured order to be lower than expected and would be an early sign that something is wrong.

Problems encountered

One issue we encountered was handling Simpson's rule when N was even. The standard Simpson's 1/3 rule requires an odd number of nodes, so we used Simpson's 3/8 rule on the final three subintervals. We then checked the measured convergence order to make sure the combined method still behaved as expected.

Full test suite

```
runTests
```

```
EECE 306 toolbox test suite
```

```
-----
```

```
eta =
```

```
376.7303
```

```
test_lab01 checks complete
```

```
PASS test_lab01          0.05 s
```

```
test_lab02 checks complete
```

```
PASS test_lab02          0.03 s
```

```
5000-point field evaluation: 0.0006 s
```

```
test_lab03 checks complete
```

```
PASS test_lab03          0.04 s
```

```
test_lab04 checks complete
```

```
PASS test_lab04          0.05 s
```

```
-----
```

```
4 passed, 0 failed, 4 total
```

```
All tests passing.
```